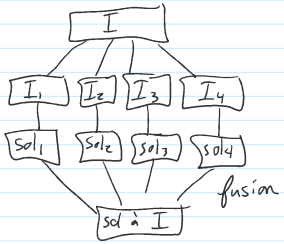
Diviser - pour - régner

5 juillet 2022 10:55



Idée: ① Séparer l'entrée en sous-instances de taille $\approx n/k$ $k \in O(1)$
② Résoudre chaque partie récursivement
③ Combiner les solutions aux sous-instances

On représente les temps de ces algos par une récurrence.

ex tri fusion: $f(n) \approx \begin{cases} O(1) & \text{si } n \leq 1 \\ 2f(\frac{n}{2}) + O(n) & \text{si } n > 1 \end{cases}$

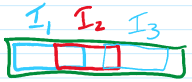
qui est $\Theta(n \log n)$

Squelette général

dpr(I)

si $|I| \leq t$, casterminal(I) // $t \in O(1)$

$I_1, I_2, I_3, \ldots, I_a$ = parties de I de taille n/b



pour chaque I_j, j = 1 à a
sol_j = dpr(I_j)

return combiner($sol_1, sol_2, \ldots, sol_j$) // temps $O(g(n))$

Ce squelette mène à la récurrence

$f(n) \approx \begin{cases} O(1) & \text{si } n \leq t \\ a \cdot f(\frac{n}{b}) + O(g(n)) & \text{si } n > t \end{cases}$

---

ex: 2 points + proche, version 1D

2 4 5 7 9 11 14

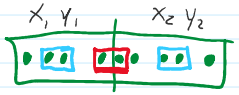


Entrée: points $P = [p_1, p_2, \dots, p_n]$ triés
Sortie: (p_i, p_j) avec $i \neq j$ qui minimise $|p_i - p_j|$

ptsProches(P) // P trié

si $|P| \leq 1$, return nil

si $|P| == 2$, return (p_1, p_2)



$(x_1, y_1) = ptsProches(P[1 .. \lfloor n/2 \rfloor])$ $f(\frac{n}{2})$

$(x_2, y_2) = ptsProches(P[\lfloor n/2 \rfloor + 1 .. n])$ $f(\frac{n}{2})$

$O(1)$ sans compter les récursions

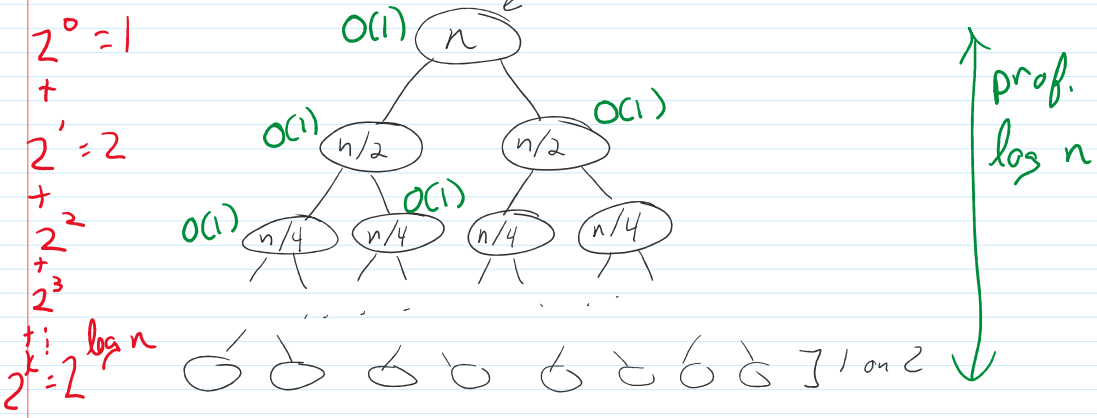
$(x_3, y_3) = (P[\lfloor \frac{n}{2} \rfloor], P[\lfloor \frac{n}{2} \rfloor + 1])$

$O(1)$

retourner la paire (x_i, y_i) parmi $\{(x_1, y_1), (x_2, y_2), (x_3, y_3)\}$ à distance min.

Récurrence: $$f(n) = \begin{cases} O(1) & \text{si } n \leq 2 \\ 2f(\frac{n}{2}) + O(1) & \text{si } n > 2 \end{cases}$$

Arbre de récursion — appel initial



$2^0 = 1$ + $2^1 = 2$ + 2^2 + 2^3 + ... + $2^{\log n} = 2^k$

Complexité: $O(1)$ × nb de noeuds de cet arbre

Complexité : $O(1) \times$ nb de noeuds de cet arbre

$2^0 + 2^1 + 2^2 + \dots + 2^{\log n}$

$$\sum_{i=0}^{\log n} 2^i = 2^{\log n + 1} - 1$$

$$= 2 \cdot 2^{\log n} - 1$$

$$= 2 \cdot n - 1 \in O(n)$$

Multiplier de grands entiers

Entrée : entiers x et y, base 10 avec n chiffres chacun

Sortie : $x * y$

Algo du primaire

X	2	0	2	2
Y	0	4	5	6
	12	1	3	2
	60	6	6	0
	80	8	0	0
	88	1	5	9	2

Temps $\Theta(n^2)$

Algo de Karatsuba

On suppose que $n = 2^k$ (sinon on ajoute des 0)

On sépare x en 2 parties x_1 et x_2
$x_1 = \frac{n}{2}$ premiers chiffres de x
$x_2 = \frac{n}{2}$ derniers " de x

On sépare y en y_1 et y_2 de la même façon

$x = 12345678$
$y = 33466791$
$x_1 = 1234 \quad x_2 = 5678$
$y_1 = 3346 \quad y_2 = 6791$

On remarque que $x = 10^{n/2} \cdot x_1 + x_2$

$y = 10^{n/2} \cdot y_1 + y_2$

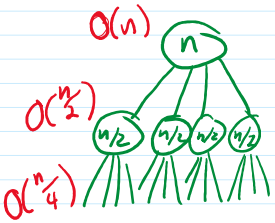
Donc $x \cdot y = (10^{n/2} x_1 + x_2)(10^{n/2} y_1 + y_2)$

$= 10^n x_1 y_1 + 10^{n/2} x_1 y_2 + 10^{n/2} x_2 y_1 + x_2 y_2$

$10^{n/2}(x_1 y_2 + x_2 y_1)$

Idée: calculer récursivement $x_1 y_1$, $x_1 y_2$, $x_2 y_1$, $x_2 y_2$

Faire les 10^n et les $+$ en temps $O(n)$



Récurrence de l'algo sous-jacent : $f(n) = \begin{cases} O(1) & \text{si } n \leq 1 \\ 4f(\frac{n}{2}) + O(n) & \text{si } n > 1 \end{cases}$

Ceci est $\Theta(n^2)$

On utile le fait que

→ $x_1 y_2 + x_2 y_1 = (x_1 + x_2)(y_1 + y_2) - x_1 y_1 - x_2 y_2$

$x \cdot y = 10^n x_1 y_1 + 10^{n/2}(x_1 y_2 + x_2 y_1) + x_2 y_2$

$= 10^n x_1 y_1 + 10^{n/2}((x_1 + x_2)(y_1 + y_2) - x_1 y_1 - x_2 y_2) + x_2 y_2$

M_1 M_2 déjà fait M_1 M_3 déjà fait M_3

M_1 M_2 déjà fait M_1 M_3 déjà fait M_3

karat(x, y) //n chiffres
si $n=1$, return $x \cdot y$ //$O(1)$

$O(n)$ | calculer x_1, x_2, y_1, y_2
$M_1 = \text{karat}(x_1, y_1)$
$O(n)$ | $t_1 = x_1 + x_2$
$t_2 = y_1 + y_2$
$M_2 = \text{karat}(t_1, t_2)$
$M_3 = \text{karat}(x_2, y_2)$
$O(n)$ | return $10^n * M_1 + 10^{n/2} * (M_2 - M_1 - M_3) + M_3$

Récurrence : $f(n) = \begin{cases} O(1) & \text{si } n=1 \\ 3 \cdot f(\frac{n}{2}) + O(n) & \text{si } n>1 \end{cases}$

ceci est $\Theta(n^{\log_2 3}) \approx O(n^{1.59})$

Théorème maître

Si vous avez une récurrence de la forme

$$f(n) = \begin{cases} O(1) & \text{si } n \leq t \quad t \in O(1) \\ a \cdot f(\frac{n}{b}) + O(g(n)) & \text{si } n > t \end{cases}$$

$\log^k n = (\log n)^k$

a = # de divisions (# d'appels)

b = diviseur de l'instance

alors

① si $g(n) = n^c$ avec $c \geq 0$ et $\log_b a > c$, alors $f(n) \in O(n^{\log_b a})$

avec $c \geq 0$

② si $g(n) = n^c \log^k n$ et $\log_b a = c$, alors $f(n) \in O(n^c \log^{k+1} n)$
avec $c \geq 0$, $k \geq 0$

③ si $g(n) = n^c$ et $\log_b a < c$, alors $f(n) \in O(n^c)$

↳ $f(n) = 2f(n/2) + O(n^0)$

$g(n)$ = complexité de la phase "combiner" de l'instance

---

tri fusion: $f(n) = 2f(n/2) + O(n)$ $\quad a=2 \quad b=2 \quad g(n) = n = n^1 \quad c=1$

$\log_b a = \log_2 2 = 1 = c$

$k=0$

car $\log^0 n = 1$

$\Rightarrow$ cas ②: $O(n^c \log^{k+1} n) = O(n \log n)$

pts Proches 1D: $f(n) = 2f(n/2) + O(1)$ $\quad (c \; k)$

$a=2 \quad b=2 \quad g(n) = 1 = n^0 \cdot \log^0 n$

$\log_a b = 1 > c = 0$

$\Rightarrow$ cas ①: $O(n^{\log_2 2}) = O(n)$

karatsuba: $f(n) = 3 \cdot f(n/2) + O(n)$

$a=3 \quad b=2 \quad c=1$

karatsuba : $f(n) = 3 \cdot f(n/2) + O(n)$

$a = 3 \quad b = 2 \quad c = 1$

$\log_2 3 > 1$

cas ① $\Rightarrow O(n^{\log_2 3}) \approx O(n^{1.59})$

fouille dicho : si $|T| = 1$, $(O(1))$
sinon
récursion dans la bonne moitié

$f(n) = 1 \cdot f(n/2) + O(1)$

$a = 1 \quad b = 2 \quad g(n) = 1 = n^0 \log^0 n$

$c = 0 \quad k = 0$

$\log_b a = \log_2 1 = 0$

$c = 0$

$\Rightarrow$ cas ② : $O(n^0 \cdot \log^{0+1} n) = O(\log n)$

algoBidon : algo(T)

```
A = algo( T[1 .. n/3] )
B = algo( T[n/3+1 .. 2n/3] )
C = algo( T[2n/3+1 .. n] )
trifusion(A+B+C)
return
```

A, B, C : $3 f(n/3)$

trifusion : $O(n \log n)$

$f(n) = 3 f(n/3) + O(n \log n)$

$a = 3 \quad b = 3 \quad c = 1 \quad k = 1 \quad g(n) = n^1 \log^1 n$

$\log_b a = \log_3 3 = 1 = c$

caso (2) $\Rightarrow O(n^c \log^{k+1} n) = O(n \log^2 n)$